



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

PROGRAMACIÓN NATIVA

“REPORTE DEL CURSO: INTRODUCTION TO FLUTTER”

PRESENTA:

RIVERA MARTINEZ JESÚS

N. DE CONTROL:

22350405

SEMESTRE: 8

GRUPO: A

DOCENTE:

JULIO AGUILAR CARMONA

ESPECIALIDAD:

INGENIERIA INFORMÁTICA

FECHA DE ENTREGA:

11/JUNIO/2026



Contenido

INTRODUCCIÓN	1
DESARROLLO DE LOS TEMAS DEL CURSO	2
1. Introducción al curso	2
2. Configuración del entorno de desarrollo	2
3. Creación de una interfaz básica	3
4. Creación de clases para organizar el código	3
5. Agregar lógica al diseño.....	4
6. Uso de widgets comunes.....	5
7. Colores, contenedores, gestos, Snackbar y gradientes.....	5
8. Text, Row, Column y SingleChildScrollView	6
9. Flexible y Expanded	7
10. ListTile, ListView y Padding	7
11. Widgets Stateful y Stateless	8
12. Formularios.....	8
13. Validación de formularios.....	9
14. Envío de formularios.....	9
15. Navegación entre pantallas	10
16. Función push().....	10
17. Función pop().....	11
18. Funciones de reemplazo y limpieza de rutas.....	11
19. Manejo de datos JSON.....	12
20. JSON con arreglos	12
21. JSON anidado	13
22. Proyecto final: Earthquake App.....	13
APRENDIZAJES OBTENIDOS	14
CONCLUSIÓN	15
ANEXOS	16

INTRODUCCIÓN

El presente reporte describe los temas abordados durante el curso “Introduction to Flutter”, disponible en la plataforma Udemty. El curso estuvo orientado a introducir los conocimientos básicos necesarios para comenzar en el desarrollo de aplicaciones móviles utilizando Flutter y el lenguaje Dart.

Durante el curso se trabajó con Visual Studio Code como entorno de desarrollo, lo cual permitió escribir, organizar y ejecutar el código de las prácticas. A través de los diferentes temas se fueron construyendo conocimientos relacionados con la creación de interfaces, el uso de widgets, la navegación entre pantallas, el manejo de formularios, la lectura de datos JSON y la construcción de una aplicación completa.

DESARROLLO DE LOS TEMAS DEL CURSO

1. Introducción al curso

En esta primera parte se presentó el propósito general del curso y se explicó la importancia de Flutter dentro del desarrollo de aplicaciones móviles. Se mencionó que Flutter permite crear aplicaciones con interfaces visuales modernas utilizando un mismo código base.

Este tema sirvió para conocer el enfoque del curso y entender que Flutter trabaja principalmente con widgets, los cuales son los componentes que forman la estructura visual de una aplicación. También permitió identificar que el curso estaba dirigido a personas principiantes interesadas en aprender los fundamentos del desarrollo móvil.

Ejemplo aplicado: Se presentó la idea general de crear aplicaciones móviles con Flutter, entendiendo que una app puede estar formada por pantallas, botones, textos, listas, formularios y elementos de navegación.

2. Configuración del entorno de desarrollo

En esta sección se abordó la instalación y preparación del entorno necesario para trabajar con Flutter. Este paso fue fundamental porque antes de programar es necesario contar con las herramientas adecuadas para crear, ejecutar y probar aplicaciones.

Se utilizó Visual Studio Code como editor de código, ya que permite trabajar con proyectos Flutter de manera organizada. Desde este entorno se pueden abrir carpetas del proyecto, editar archivos Dart, utilizar la terminal integrada y ejecutar la aplicación.

También se revisó la estructura inicial de un proyecto Flutter. Uno de los archivos más importantes es `main.dart`, ya que ahí se encuentra el punto de inicio de la

aplicación. Además, se identificaron carpetas como lib, donde normalmente se escribe el código principal del proyecto.

Ejemplo aplicado: Se configuró el entorno de trabajo y se abrió un proyecto de Flutter en Visual Studio Code. Después se revisó el archivo main.dart para identificar dónde comienza la aplicación y cómo se ejecuta una interfaz básica.

3. Creación de una interfaz básica

Después de preparar el entorno, el curso continuó con la creación de una interfaz básica. Este tema trató sobre cómo estructurar visualmente una aplicación en Flutter utilizando widgets principales.

Se explicó que una interfaz en Flutter se construye de forma jerárquica, es decir, un widget puede contener a otros widgets. Por ejemplo, una pantalla puede tener una barra superior, un cuerpo principal y dentro del cuerpo se pueden colocar textos, botones, imágenes o listas.

Uno de los widgets principales utilizados fue Scaffold, el cual sirve como base para crear la estructura visual de una pantalla. También se trabajó con AppBar, que permite colocar una barra superior en la aplicación, y con widgets de contenido para mostrar información en el centro de la pantalla.

Ejemplo aplicado: Se creó una pantalla sencilla con una barra superior y un texto en el cuerpo de la aplicación. Esta práctica permitió comprender cómo se forma una pantalla básica en Flutter y cómo se organizan los elementos dentro de ella.

4. Creación de clases para organizar el código

En esta parte del curso se explicó la importancia de organizar el código mediante clases. Cuando una aplicación es pequeña, todo el código puede escribirse en un solo archivo, pero cuando el proyecto empieza a crecer, es necesario separar las partes para mantener un mejor orden.

El uso de clases permite dividir la aplicación en componentes más claros. Por ejemplo, se puede crear una clase para una pantalla, otra para un formulario y otra para representar datos. Esto facilita la lectura del código y permite modificar una parte del proyecto sin afectar directamente a las demás.

Este tema fue importante porque enseñó una buena práctica de programación: mantener el código limpio, organizado y fácil de entender.

Ejemplo aplicado: Se separó una parte de la interfaz en una clase propia. En lugar de tener todo el diseño dentro del archivo principal, se creó una clase para representar una pantalla o componente, haciendo que el proyecto quedara más ordenado.

5. Agregar lógica al diseño

En este tema se explicó cómo agregar lógica al código de la aplicación. La lógica permite que la app no solo muestre elementos visuales, sino que también responda a las acciones del usuario.

Por ejemplo, una aplicación puede cambiar un texto cuando se presiona un botón, mostrar un mensaje, validar datos o modificar información en pantalla. Esta parte es esencial porque una app real necesita interacción, no solamente diseño.

También se comprendió que la parte visual y la parte lógica trabajan juntas. Los widgets muestran la información, pero la lógica determina qué sucede cuando el usuario interactúa con ellos.

Ejemplo aplicado: Se agregó una acción a un botón para que, al presionarlo, ocurriera un cambio dentro de la aplicación. Esta práctica ayudó a comprender cómo se relacionan los eventos del usuario con el comportamiento de la app.

6. Uso de widgets comunes

Una de las secciones más importantes del curso fue el uso de widgets comunes en Flutter. Los widgets son la base de Flutter, ya que todo lo que aparece en pantalla se construye mediante ellos.

Se trabajó con widgets como AppBar, Icon, Center, Container, Text, Row, Column, SingleChildScrollView, ListView, ListTile y Padding. Cada uno tiene una función específica dentro del diseño de la aplicación.

El widget Text permite mostrar texto en pantalla. Container sirve para agrupar elementos, aplicar tamaños, colores o márgenes. Row organiza elementos de manera horizontal, mientras que Column los acomoda de forma vertical. Center permite centrar elementos y Padding agrega espacios alrededor de los widgets.

También se revisaron widgets para listas, como ListView y ListTile, que permiten mostrar varios elementos de forma ordenada y desplazable.

Ejemplo aplicado: Se diseñó una pantalla con varios elementos organizados. Se usó Column para colocar información de arriba hacia abajo, Row para acomodar elementos en una misma línea, Container para dar estilo visual y Padding para separar los componentes.

7. Colores, contenedores, gestos, Snackbar y gradientes

Dentro del tema de widgets, también se trabajó con elementos visuales y de interacción. Esta parte trató sobre cómo mejorar la apariencia de una aplicación y cómo hacer que responda a ciertas acciones del usuario.

Se usaron colores para personalizar los elementos de la interfaz. También se utilizaron contenedores para agrupar contenido y aplicar propiedades visuales, como tamaño, color, bordes o fondos.

Los detectores de gestos permitieron identificar acciones del usuario, como tocar un elemento de la pantalla. Por otra parte, el Snackbar permitió mostrar mensajes

temporales dentro de la aplicación, por ejemplo, una notificación breve después de presionar un botón.

Los gradientes se utilizaron para dar un mejor diseño visual, aplicando transiciones de color en los fondos o contenedores.

Ejemplo aplicado: Se creó un contenedor con color o degradado y se le agregó una acción. Al tocarlo, se mostraba un mensaje temporal mediante un Snackbar. Esta práctica permitió combinar diseño visual e interacción.

8. Text, Row, Column y SingleChildScrollView

En esta sección se profundizó en widgets utilizados para organizar contenido dentro de una pantalla. El widget Text se utilizó para mostrar cadenas de texto. Row permitió acomodar elementos horizontalmente y Column permitió organizarlos verticalmente.

También se trabajó con SingleChildScrollView, que sirve para permitir desplazamiento cuando el contenido no cabe completamente en la pantalla. Esto es útil en aplicaciones donde se muestra mucha información, ya que evita que los elementos se corten o se salgan del espacio visible.

Este tema fue importante porque enseñó a distribuir correctamente los elementos de una interfaz, considerando el tamaño de la pantalla del dispositivo.

Ejemplo aplicado: Se creó una pantalla con varios textos y componentes organizados en una columna. Como el contenido podía ocupar más espacio del disponible, se utilizó SingleChildScrollView para permitir que el usuario pudiera desplazarse hacia abajo.

9. Flexible y Expanded

En esta parte del curso se abordó la diferencia entre Flexible y Expanded. Ambos widgets sirven para controlar cómo se distribuye el espacio dentro de una fila o columna.

Expanded hace que un widget ocupe todo el espacio disponible dentro de su contenedor principal. En cambio, Flexible permite que el widget se adapte al espacio disponible, pero con mayor flexibilidad sobre cuánto espacio puede ocupar.

Este tema es importante porque ayuda a crear interfaces que se adapten mejor a diferentes tamaños de pantalla. En desarrollo móvil, esto es necesario porque los dispositivos pueden tener dimensiones distintas.

Ejemplo aplicado: Se colocaron varios elementos dentro de una fila o columna y se utilizó Expanded para que uno de ellos ocupara el espacio restante. También se analizó cómo Flexible permite ajustar el tamaño de los elementos sin forzar completamente su expansión.

10. ListTile, ListView y Padding

Este tema trató sobre la creación de listas en Flutter. Las listas son comunes en aplicaciones móviles porque permiten mostrar información repetida, como contactos, productos, mensajes, noticias o registros.

ListView permite crear una lista desplazable. ListTile representa cada elemento de esa lista y puede incluir título, subtítulo, ícono al inicio o elemento al final. Padding se utiliza para agregar espacio alrededor de los componentes y mejorar la presentación visual.

Este tema fue importante porque mostró cómo representar varios datos en pantalla de una forma organizada y fácil de leer.

Ejemplo aplicado: Se creó una lista de elementos usando ListView. Cada elemento fue diseñado con ListTile, mostrando un ícono, un título y una descripción. También se agregó Padding para separar visualmente los elementos.

11. Widgets Stateful y Stateless

En esta sección se explicó la diferencia entre widgets Stateful y Stateless. Esta diferencia es fundamental en Flutter.

Un widget Stateless se utiliza cuando la información que muestra no cambia durante la ejecución. Por ejemplo, un título fijo, una etiqueta o una pantalla informativa pueden ser Stateless.

Un widget Stateful, en cambio, se utiliza cuando la información puede cambiar. Por ejemplo, un contador, un formulario, un botón que cambia contenido o una pantalla que actualiza datos requieren manejar estado.

Este tema permitió comprender cómo Flutter actualiza la interfaz cuando cambian los datos.

Ejemplo aplicado: Se creó una pantalla donde un valor cambiaba al presionar un botón. Para lograr que el cambio se reflejara en pantalla, fue necesario utilizar un widget Stateful.

12. Formularios

En esta parte del curso se trabajó con formularios, los cuales permiten capturar datos ingresados por el usuario. Los formularios son muy utilizados en aplicaciones reales, por ejemplo, para iniciar sesión, registrarse, enviar comentarios o llenar información personal.

Se revisaron elementos de formulario, campos de texto, validación y envío de datos. La validación es importante porque permite verificar que la información ingresada sea correcta antes de procesarla.

También se explicó que un formulario debe controlar los datos que introduce el usuario y mostrar advertencias cuando algo no cumple con las condiciones establecidas.

Ejemplo aplicado: Se creó un formulario con campos de texto. Después se agregó una validación para evitar que los campos quedaran vacíos. Finalmente, se programó una acción para enviar o procesar la información ingresada.

13. Validación de formularios

La validación de formularios fue un tema específico dentro del trabajo con datos del usuario. Esta parte consistió en comprobar que los datos ingresados fueran válidos antes de continuar.

Por ejemplo, se puede validar que un campo no esté vacío, que un correo tenga formato correcto o que una contraseña cumpla con cierta longitud. Esto mejora el funcionamiento de la aplicación y evita errores.

Este tema fue importante porque mostró cómo dar retroalimentación al usuario cuando la información ingresada no es correcta.

Ejemplo aplicado: Se agregó una condición para verificar que el usuario escribiera información en un campo de texto. Si el campo estaba vacío, la aplicación mostraba un mensaje de error indicando que debía completarse.

14. Envío de formularios

Después de validar los datos, se trabajó con el envío de formularios. Esta parte consistió en procesar la información ingresada por el usuario después de comprobar que era válida.

El envío puede representar diferentes acciones, como guardar datos, mostrar un mensaje, avanzar a otra pantalla o preparar la información para enviarla a un servicio externo.

Este tema ayudó a comprender el flujo básico de un formulario: ingresar datos, validarlos y después enviarlos o procesarlos.

Ejemplo aplicado: Después de llenar correctamente los campos del formulario, se presionó un botón para enviar la información. La aplicación procesaba los datos y podía mostrar un mensaje confirmando que el formulario había sido enviado.

15. Navegación entre pantallas

En esta sección se explicó cómo moverse entre diferentes pantallas dentro de una aplicación Flutter. La navegación es fundamental porque la mayoría de las aplicaciones no tienen una sola pantalla, sino varias secciones.

Se trabajó con funciones como `push`, `pop`, `pushReplacementNamed`, `popAndPushNamed`, `popUntil` y `pushNamedAndRemoveUntil`. Estas funciones permiten controlar cómo entra y sale el usuario de las pantallas.

`push` permite abrir una nueva pantalla. `pop` permite regresar a la anterior. Otras funciones permiten reemplazar pantallas o eliminar rutas anteriores para controlar mejor el historial de navegación.

Ejemplo aplicado: Se creó una aplicación con una pantalla principal y una pantalla secundaria. Desde la pantalla principal se presionaba un botón para ir a la segunda pantalla. Después, usando otra acción, se podía regresar a la pantalla anterior.

16. Función `push()`

La función `push()` se utilizó para enviar al usuario a una nueva pantalla. Esta función agrega una pantalla encima de la actual, formando una pila de navegación.

Este concepto es parecido a abrir una nueva página dentro de la aplicación. La pantalla anterior no desaparece, sino que queda debajo, permitiendo regresar a ella posteriormente.

Ejemplo aplicado: Desde una pantalla principal se presionó un botón y la aplicación abrió una nueva pantalla con más información.

17. Función pop()

La función pop() se utilizó para regresar a la pantalla anterior. Esta función elimina la pantalla actual de la pila de navegación y muestra nuevamente la pantalla previa.

También se revisó el manejo del botón de retroceso y la posibilidad de confirmar una acción antes de salir de una pantalla.

Ejemplo aplicado: En una pantalla secundaria se agregó un botón para regresar a la pantalla principal. Al presionarlo, la aplicación cerraba la pantalla actual y volvía a la anterior.

18. Funciones de reemplazo y limpieza de rutas

Además de push() y pop(), se revisaron funciones más avanzadas de navegación. pushReplacementNamed() permite reemplazar la pantalla actual por otra. popAndPushNamed() permite quitar una pantalla y abrir otra nueva.

También se revisaron popUntil() y pushNamedAndRemoveUntil(), que sirven para regresar hasta una pantalla específica o eliminar varias pantallas del historial de navegación.

Estas funciones son útiles en aplicaciones donde no se desea que el usuario regrese a pantallas anteriores, por ejemplo, después de iniciar sesión o después de cerrar sesión.

Ejemplo aplicado: Se simuló una navegación donde, después de avanzar a cierta pantalla, se eliminaban rutas anteriores para evitar que el usuario regresara a una sección que ya no debía estar disponible.

19. Manejo de datos JSON

En esta sección se trabajó con datos en formato JSON. JSON es un formato muy utilizado para intercambiar información entre aplicaciones y servicios web.

El curso abordó JSON simple, JSON con arreglos y JSON anidado. Esto permitió comprender diferentes formas en que la información puede estar organizada.

Un JSON simple puede contener datos directos, como nombre, edad o descripción. Un JSON con arreglos puede contener listas de elementos. Un JSON anidado puede contener información dentro de otros objetos, lo que requiere leer los datos con más cuidado.

Ejemplo aplicado: Se leyó información en formato JSON y se extrajeron valores específicos para mostrarlos en la aplicación. También se trabajó con listas de datos para presentarlas dentro de una interfaz.

20. JSON con arreglos

El JSON con arreglos fue importante porque muchas aplicaciones reciben listas de datos. Por ejemplo, una lista de usuarios, productos, noticias o eventos.

En este tema se aprendió que un arreglo JSON contiene varios elementos y que cada uno puede convertirse en un elemento visual dentro de la aplicación.

Ejemplo aplicado: Se procesó una lista de elementos en formato JSON y se mostró cada elemento dentro de un ListView.

21. JSON anidado

El JSON anidado se trabajó para comprender estructuras de datos más complejas. En este tipo de JSON, un objeto puede contener otros objetos o arreglos dentro de él.

Este tema fue importante porque en aplicaciones reales los datos no siempre vienen de forma simple. Muchas veces es necesario acceder a información que está dentro de varias capas.

Ejemplo aplicado: Se leyó un JSON donde algunos datos estaban dentro de otros objetos. La práctica consistió en acceder a esos valores internos y utilizarlos dentro de la aplicación.

22. Proyecto final: Earthquake App

El proyecto final del curso consistió en construir una aplicación relacionada con información de terremotos. Esta práctica integró varios temas vistos anteriormente, como interfaz gráfica, organización del código, manejo de datos, peticiones HTTP y presentación de información.

Primero se creó una interfaz básica para mostrar la información. Después se trabajó con una clase para manejar la conexión con una API. También se utilizó FutureBuilder, que permite mostrar datos que se obtienen de forma asíncrona, es decir, datos que tardan un momento en cargarse.

El proyecto también incluyó una petición HTTP para obtener información desde una fuente externa. Esto permitió entender cómo una aplicación puede conectarse a internet, solicitar datos y mostrarlos al usuario.

Ejemplo aplicado: Se desarrolló una aplicación que consultaba información sobre terremotos y mostraba los datos dentro de una interfaz. La app utilizaba una petición HTTP para obtener la información y después la presentaba visualmente mediante widgets.

APRENDIZAJES OBTENIDOS

Al finalizar el curso, se obtuvo una comprensión general de cómo iniciar un proyecto en Flutter y cómo construir aplicaciones móviles básicas. Se aprendió a configurar el entorno de trabajo, utilizar Visual Studio Code, crear interfaces mediante widgets, organizar el código en clases, agregar lógica, trabajar con formularios, navegar entre pantallas y leer datos JSON.

También se comprendió que Flutter permite desarrollar aplicaciones de forma estructurada. Primero se crea la interfaz, después se agrega interacción, luego se manejan datos y finalmente se integran todos los elementos en una aplicación funcional.

CONCLUSIÓN

El curso “Introduction to Flutter” permitió conocer los fundamentos del desarrollo de aplicaciones móviles utilizando Flutter. A través de sus temas y prácticas se logró comprender cómo se construyen interfaces, cómo se organizan los elementos en pantalla, cómo se agregan acciones mediante lógica y cómo se trabaja con datos externos.

El uso de Visual Studio Code facilitó el desarrollo de las prácticas, ya que permitió escribir y organizar el código de manera clara. Además, el proyecto final de la aplicación de terremotos ayudó a integrar los conocimientos obtenidos durante el curso, demostrando cómo Flutter puede utilizarse para crear aplicaciones más completas y funcionales.

En conclusión, el curso fue útil como introducción al desarrollo móvil, ya que presentó los conceptos básicos de Flutter mediante ejemplos prácticos y permitió comprender el proceso de creación de una aplicación desde sus primeros pasos hasta un proyecto final.

ANEXOS

The screenshot shows the Udemy website interface. At the top, the navigation bar includes the Udemy logo, links for 'Explorar' and 'Suscribirse', a search bar, and links for 'Udemy Business', 'Enseña en Udemy', and 'Mi aprendizaje'. The main header area is dark blue with the text 'Mi aprendizaje' and a sub-header with links: 'Todos los cursos', 'Mis listas', 'Lista de deseos', 'Vías de aprendizaje', 'Archivados', and 'Herramientas de aprendizaje'. The main content area features a 'Racha semanal' (Weekly Streak) section with a purple flame icon, indicating a 1-week streak. Below this is a course card for 'Introduction to Flutter' by DSC VIT, showing a 100% completion progress bar and a star rating. On the right, a user profile sidebar for 'Jesús Rivera Martínez' (username: jesusrm519@gmail.com) is visible, containing links to 'Mi aprendizaje', 'Mi cesta', 'Lista de deseos', 'Enseña en Udemy', 'Notificaciones', 'Mensajes', 'Configuración de la cuenta', 'Métodos de pago', 'Suscripciones', 'Créditos de Udemy', 'Historial de compras', 'Idioma' (set to Español), and 'Perfil público'.

udemy.com/home/my-courses/learning/

udemy Explorar Suscribirse Buscar cualquier cosa Udemy Business Enseña en Udemy Mi aprendizaje

Mi aprendizaje

Todos los cursos Mis listas Lista de deseos Vías de aprendizaje Archivados Herramientas de aprendizaje

Racha semanal
Estás avanzando bien para conseguir tus objetivos de aprendizaje.

1 semana
Racha actual

0/30 minutos
1/1 visita
3 may - 9

Introduction to Flutter
DSC VIT Powered by Google Developers, Satkriti Singh...

100 % completado
★★★★☆
Tu calificación

<https://www.udemy.com/user/edit-profile/>

JM Jesús Rivera Martínez
jesusrm519@gmail.com

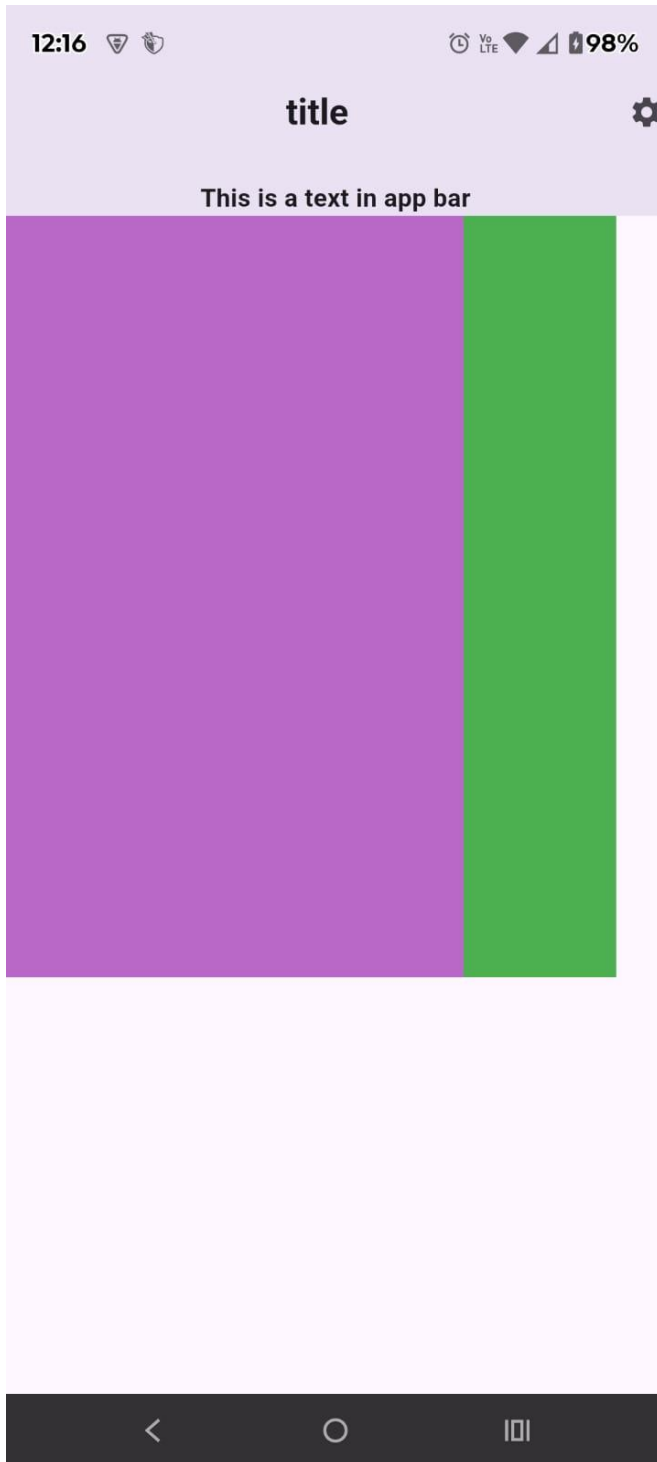
- Mi aprendizaje
- Mi cesta
- Lista de deseos
- Enseña en Udemy
- Notificaciones
- Mensajes
- Configuración de la cuenta
- Métodos de pago
- Suscripciones
- Créditos de Udemy
- Historial de compras
- Idioma: Español
- Perfil público

Screen 2



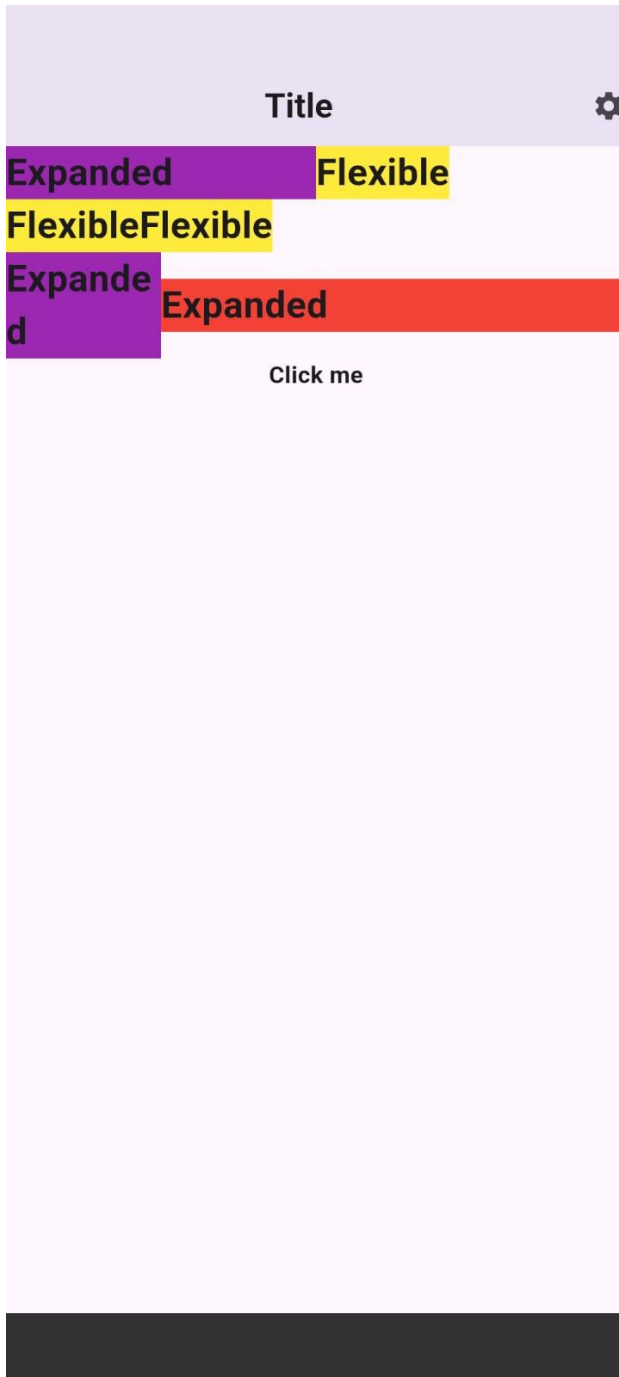
Screen 2
Take me back!

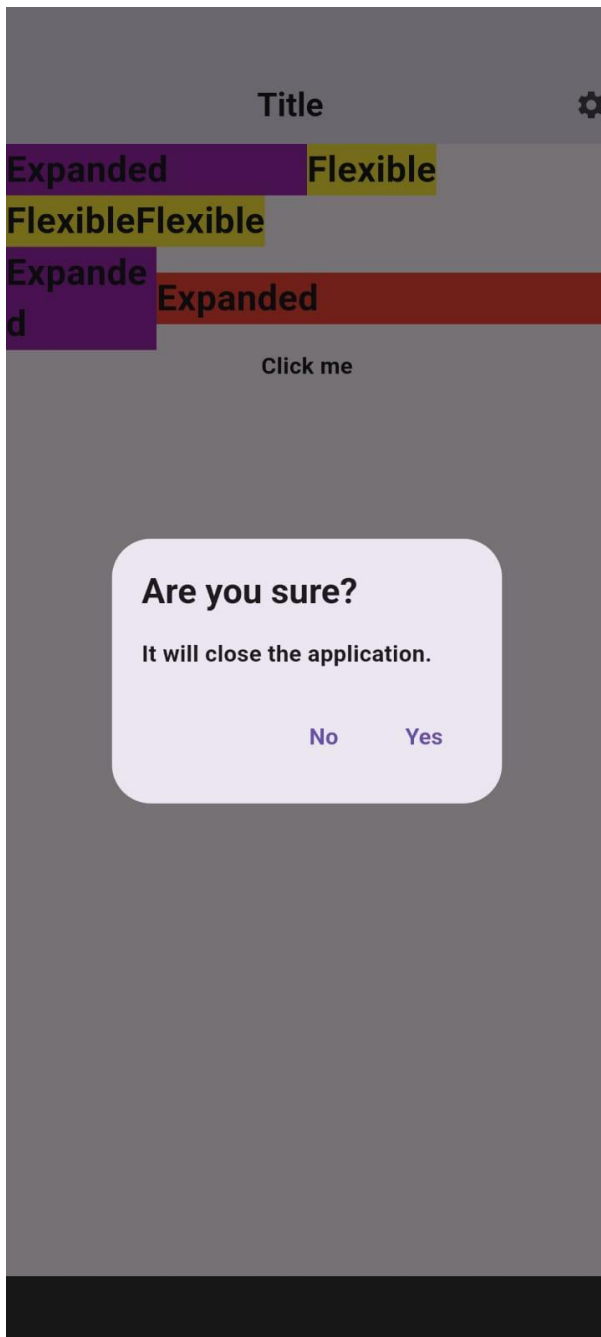


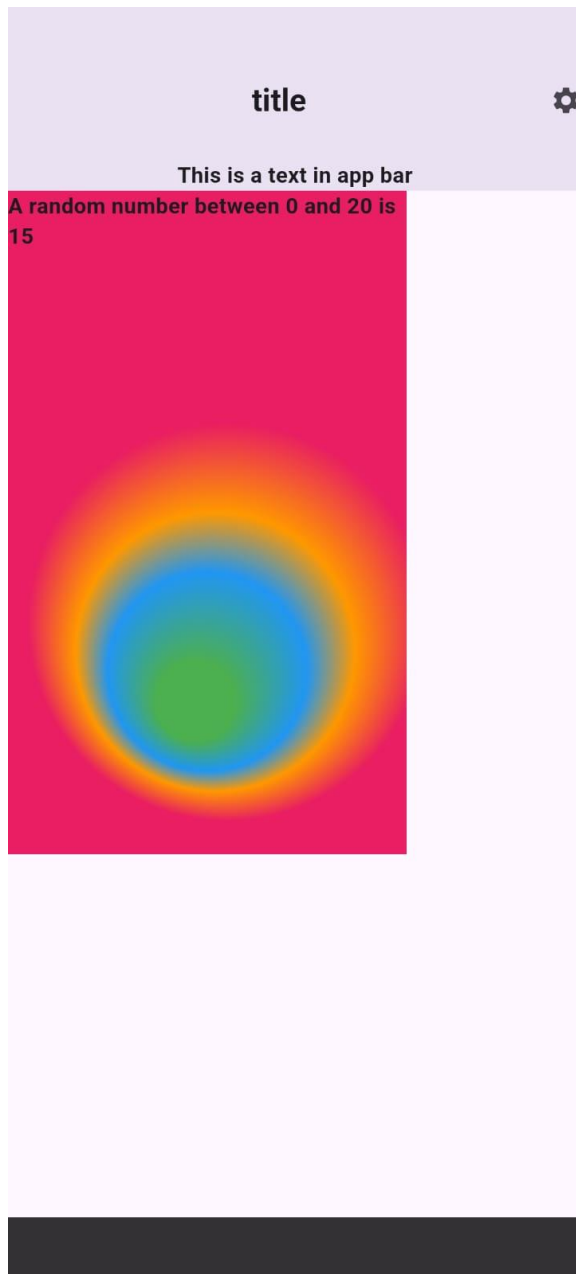












Title

Enter Name

jesus R

Enter Age

12

Enter Password

.....

B

☒ Single

☐ Married

☒ Sign up for the newspaper and related articles

Register

Title

Enter Name

Please enter a name

Enter Age

Please enter your age

Enter Password

Please enter a password

Please choose the city you live in

☒ Single

☐ Married

☐ Sign up for the newspaper and related articles

Register

Wheather

2

CA

10km SSW of Idyllwild

2

Central Alaska

Central Alaska

2

Alaska

7 km SSW of Big Lake

2

Alaska

32 km N of Petersville

4

Chile

28 km WNW of Coquimbo

1

California

5 km WSW of Cobb

1

California

6 km E of Mammoth Lakes

1

CA

11km ESE of Anza

1

Nevada

14 km ENE of Hawthorne

5

south of the Fiji Islands

south of the Fiji Islands

2

Alaska

74 km SW of Unalaska